

05-08 Лекция: AI-агенты, Git и GitHub

Дата и время: 2026-05-08 12:06:39

Местоположение: [Вставьте Местоположение]

Инструктор: [Вставьте Имя] / Николай

Краткое содержание

Лекция посвящена созданию, управлению и масштабированию AI-агентов в средах разработки, таких как Codex и OpenCode, а также основам работы с системой контроля версий Git и платформой GitHub. Обсуждаются ключевые проблемы при работе с AI-агентами, такие как ограничение контекстного окна моделей, и предлагаются практические решения для управления долгосрочными проектами, включая создание файлов `.md` с ролью агента и папок с «долгосрочной памятью». Рассматриваются расширенные возможности, такие как создание локальных и глобальных «скиллов» (навыков) и автоматизация задач с помощью «хуков». Отдельный блок посвящен системе Git: объясняются её назначение, основные концепции (репозиторий, ветки, коммиты, слияние) и команды (`push` , `pull`). Демонстрируется практическая работа: инициализация репозитория, связывание локальной среды с GitHub и публикация проекта в удаленном репозитории. Подчеркивается важность GitHub для версионирования, синхронизации работы, резервного копирования и совместной разработки.

Основные положения

1. Создание и управление AI-агентами

- **Создание и конфигурация агента:**
 - Чтобы создать нового агента, достаточно создать в папке проекта файл `Agents.md` (название должно быть написано заглавными буквами) и описать в нем его роль и задачи.
 - Системный промпт сильно влияет на поведение агента, и он не может полностью забыть свою базовую сущность (например, что он — Codex). Для более глубокой настройки можно редактировать файл конфигурации агента.

- При взаимодействии с агентом следует давать конкретные инструкции, например, явно указывать на использование официальной документации для выполнения задач.
- **Проблема ограниченного контекста:**
 - Контекстное окно языковых моделей ограничено (например, 256 тыс. токенов у GPT, до 12 млн. у тестовых моделей). С каждым сообщением агент загружает весь предыдущий контекст, что быстро расходует лимит токенов.
 - Когда лимит достигается, агент сжимает контекст, что может привести к потере информации.
- **Управление долгосрочными проектами и «долгосрочная память»:**
 - Для сложных проектов рекомендуется создавать для агента «долгосрочную память» в виде папок и файлов, включающих:
 - **Бэклог (backlog):** Список запланированных задач.
 - **Список выполненных задач.**
 - **Актуальная документация:** Агенту можно поручить документировать создаваемые им функции.
 - При запуске новой сессии агент обращается к этим файлам, что позволяет ему понимать текущее состояние проекта и предотвращает потерю информации.

2. Расширенные возможности и автоматизация в Codex

- **Локальные и глобальные скиллы (навыки):**
 - **Скиллы** — это предопределенные действия для автоматизации повторяющихся задач.
 - **Локальные скиллы** работают только в пределах конкретного проекта (папки).
 - **Глобальные скиллы** доступны во всех проектах и хранятся в папке `.codex_skills`. По умолчанию скиллы создаются как глобальные.
- **Хуки (Hooks):**
 - **Хуки** — это триггеры, которые автоматически запускают команды или скрипты при определенных событиях (например, до или после сообщения). Это более жесткий механизм автоматизации, чем `Agents.md`.

- Примеры: автоматическое обновление памяти агента, открытие дашборда при запуске, автоматизация работы с Kanban-доской или создание Git-коммитов.

- **Автоматизация (Automations):**

- Вкладка “Automations” в Codex позволяет настраивать задачи по расписанию, но требует, чтобы компьютер и приложение были постоянно запущены. Для критически важных задач рекомендуется использовать другие инструменты, например, Open Core.

3. Основы системы контроля версий Git и GitHub

- **Назначение и основные концепции Git:**

- **Git** — система контроля версий для отслеживания изменений в коде. Она сохраняет не только текущее состояние проекта, но и всю историю изменений (версионирование).
- **Коммит (Commit):** «Снимок» состояния проекта в определенный момент, точка сохранения с описанием внесенных изменений.
- **Ветки (Branches):** Позволяют изолированно работать над новыми функциями (`фичами`), не затрагивая основную, стабильную версию кода (`main`). После завершения работы ветка сливается (`merge`) с основной.

- **GitHub как облачная платформа:**

- **GitHub** — это облачное хранилище для Git-репозиториях.
- **Репозиторий** — аналог папки проекта в облаке, где хранится весь код и его история.
- **Push:** Команда для отправки локальных изменений (коммитов) в удаленный репозиторий на GitHub.
- **Pull:** Команда для загрузки актуальной версии кода из удаленного репозитория на локальную машину.

- **Преимущества использования Git и GitHub:**

- **Синхронизация:** Позволяет работать над одним проектом с разных устройств или в разных средах, поддерживая код в актуальном состоянии.
- **Резервное копирование:** Код хранится удаленно, что защищает от потери данных.
- **Совместная работа:** Упрощает коллаборацию, позволяя нескольким разработчикам или агентам работать над одним проектом.

- **Организации:** На GitHub можно создавать «организации» для централизованного управления репозиториями компании.

4. Практика и лучшие практики

- **Организация проекта:** Каждый проект должен находиться в своей отдельной папке для удобства управления.
- **Хранение секретной информации:** Ключи доступа и пароли следует хранить в специальном файле `.env`, а не в коде или `Agents.md`. Это критически важно для безопасности.
- **Работа с Git в CodeX:**
 - **Инициализация:** Для начала работы с Git в папке проекта нужно выполнить команду `git init`.
 - **Авторизация:** Необходимо связать CodeX с учетной записью GitHub через специальный плагин для публикации проектов.
 - **Визуализация:** CodeX визуализирует изменения в файлах, упрощая создание коммитов и выполнение Git-команд.
- **Определение “Фичи” (Feature):** Это новая функция или особенность продукта (например, еженедельная отправка денег в интернет-банке).

Вопросы

- [Вставьте Вопрос/Недопонимание]

Задания

- [] 1. Создать аккаунт на GitHub.
- [] 2. При работе над долгосрочным проектом создать для AI-агента папку с “долгосрочной памятью” (бэклог, выполненные задачи) и поручить ему вести документацию.
- [] 3. Использовать файл `.env` для хранения ключей доступа и другой конфиденциальной информации.
- [] 4. Создать локально тестовый проект, инициализировать в нем Git.
- [] 5. Создать публичный репозиторий на GitHub и опубликовать (запустить) в него свой проект. Поделиться ссылкой на репозиторий.
- [] 6. Попробовать создать собственный локальный или глобальный «скилл» в Codex для автоматизации рутинной задачи.
- [] 7. (Опционально) Попробовать создать «организацию» на GitHub.

- [] 8. При создании агентов в Codex давать явные инструкции для обращения к официальной документации.